

ДИСЦИПЛИНА	Технологии индустриального программирования
ИНСТИТУТ	Институт технологий и перспективных индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Адышкин Сергей Сергеевич
СЕМЕСТР	2 семестр, 2025-2026 гг.

Практическое занятие №5

Реализация HTTPS (TLS-сертификаты). Защита от SQL-инъекций

Цель занятия

Освоить базовые практические подходы к защите backend-приложения на Go за счёт включения HTTPS с использованием TLS-сертификата и предотвращения SQL-инъекций при работе с базой данных.

Задачи занятия

1. Изучить назначение HTTPS и роль TLS в защите сетевого взаимодействия.
2. Понять, чем HTTP отличается от HTTPS на уровне backend-приложения.
3. Освоить подключение TLS-сертификата к серверу на Go.
4. Научиться запускать HTTP-сервис в защищённом режиме через HTTPS.
5. Изучить природу SQL-инъекций и причины их возникновения.
6. Понять, почему конкатенация SQL-строк с пользовательским вводом опасна.
7. Освоить безопасную работу с SQL-запросами через параметризованные запросы и prepared statements.
8. Реализовать учебный пример backend-приложения с HTTPS и безопасным доступом к данным.

Теоретическая часть

1. Зачем нужен HTTPS

Обычный HTTP передаёт данные без криптографической защиты. Это означает, что содержимое запроса и ответа потенциально может быть прочитано или изменено, если канал связи не защищён. HTTPS решает эту проблему за счёт использования TLS.

В Go HTTPS-сервер обычно поднимается на базе `net/http`, а TLS-материалы передаются как сертификат и ключ. Поддержка TLS встроена в стандартную библиотеку, а `tls.LoadX509KeyPair` работает именно с PEM-данными сертификата и ключа. (pkg.go.dev)

Для backend-разработчика HTTPS — это не дополнительная опция «на будущее», а стандартный способ безопасной публикации серверного приложения.

2. Что такое TLS-сертификат

TLS-сертификат – это криптографический объект, который используется для подтверждения идентичности сервера и установления защищённого соединения. В учебной локальной среде часто используют самоподписанный сертификат, чтобы отработать механику HTTPS без внешнего удостоверяющего центра.

Команда `openssl req` предназначена прежде всего для создания и обработки `certificate requests`, но также может создавать `self-signed certificates`. Поддержка опции `-x509` и параметров вроде `-newkey`, `-keyout`, `-out`, `-days` отражена в официальной документации OpenSSL. (docs.openssl.org)

3. Что меняется в приложении при переходе с HTTP на HTTPS

На уровне логики бизнес-кода обычно ничего не меняется: маршруты, обработчики, JSON и база данных остаются теми же. Но меняется транспортный уровень — сервер начинает принимать защищённые HTTPS-соединения вместо обычного HTTP.

С точки зрения Go это означает, что сервер запускается через TLS-вариант, а приложению нужны файлы сертификата и ключа. Для загрузки пары сертификат/ключ может использоваться `tls.LoadX509KeyPair`, которая ожидает PEM-файлы. ([pkg.go.dev](https://pkg.go.dev/crypto/tls#LoadX509KeyPair))

4. Что такое SQL-инъекция

SQL-инъекция возникает тогда, когда приложение строит SQL-запрос динамически, подставляя пользовательский ввод прямо в строку запроса. OWASP указывает, что атака возможна, если приложение использует `dynamic database queries with string concatenation and user-supplied input`. В качестве основной защиты OWASP рекомендует `prepared statements / parameterized queries`, а также `allow-list validation`. ([Cheat Sheet Series](#))

Go-документация по `database/sql` также прямо предупреждает: собирать SQL через `fmt.Sprintf` небезопасно. Вместо этого нужно передавать значения параметров отдельно, чтобы пакет `sql` превращал запрос в `prepared statement` и отправлял параметр отдельно от текста SQL. Документация отдельно отмечает, что `placeholder syntax` зависит от СУБД и драйвера; для PostgreSQL драйвер `pq` использует форму вида `$1`. ([Go](#))

5. Почему `parameterized queries` безопаснее

Когда запрос выполняется как параметризованный, текст SQL и значения параметров передаются отдельно. Это позволяет СУБД

интерпретировать пользовательский ввод как данные, а не как часть SQL-синтаксиса. В Go-документации сказано, что `functions in the sql package create prepared statements from the arguments you supply`, а в `prepared statements placeholders` заполняются значениями отдельно от SQL-текста. ([Go](#))

6. Что мы сделаем в этой работе

В этой практике будет реализован учебный сервис со следующими маршрутами:

`GET /health` — проверка работы сервиса;

`GET /students?id=...` — получение студента по идентификатору.

Приложение будет запускаться по HTTPS.

Также в работе будет показано:

- как выглядит **опасный** SQL-запрос;
- как тот же запрос переписывается в **безопасном** виде;
- как использовать `prepared statement` в Go.

Практическая часть

Общая идея проекта

В этой работе создаётся приложение на Go, которое:

- подключается к PostgreSQL;
- поднимает HTTPS-сервер;
- принимает запросы по защищённому адресу;
- получает данные из базы безопасным способом.

Рекомендуемая структура проекта

```
pz5-security/  
  cmd/  
    server/  
      main.go  
  certs/  
    server.crt  
    server.key  
  internal/  
    config/  
      config.go  
    httpapi/  
      handler.go  
    student/  
      model.go  
      repo.go  
  sql/  
    init.sql  
  go.mod
```

Шаг 1. Создание проекта

Откройте PowerShell и создайте папку проекта:

```
mkdir pz5-security
```

```
cd pz5-security
go mod init example.com/pz5-security
mkdir cmd
mkdir cmd\server
mkdir certs
mkdir internal
mkdir internal\config
mkdir internal\httpapi
mkdir internal\student
mkdir sql
```

Для macOS/Linux:

```
mkdir -p pz5-security/cmd/server
mkdir -p pz5-security/certs
mkdir -p pz5-security/internal/config
mkdir -p pz5-security/internal/httpapi
mkdir -p pz5-security/internal/student
mkdir -p pz5-security/sql
cd pz5-security
go mod init example.com/pz5-security
```

Шаг 2. Подключение зависимостей

Для PostgreSQL:

```
go get github.com/lib/pq
```

В этой практике используется PostgreSQL, потому что он уже логично продолжает предыдущие темы курса по backend-разработке и SQL.

Шаг 3. Подготовка базы данных

Создайте файл `sql/init.sql`:

```
CREATE TABLE IF NOT EXISTS students (
```

```
        id BIGSERIAL PRIMARY KEY,
        full_name TEXT NOT NULL,
        study_group TEXT NOT NULL,
        email TEXT NOT NULL UNIQUE
    );

INSERT INTO students (full_name, study_group, email)
VALUES
    ('Иванов      Иван      Иванович',      'ИБВО-01-25',
'ivanov@example.com'),
    ('Петрова      Мария      Сергеевна',      'ИБВО-02-25',
'petrova@example.com'),
    ('Сидоров      Алексей      Андреевич',      'ИБВО-03-25',
'sidorov@example.com')
ON CONFLICT (email) DO NOTHING;
```

Создайте базу данных, например `study_security`, и выполните этот SQL-скрипт.

Шаг 4. Генерация учебного TLS-сертификата

Для локальной практики можно использовать самоподписанный сертификат.

Официальная документация OpenSSL указывает, что `openssl req` может создавать self-signed certificates, а `-x509` относится к режиму такого создания. (docs.openssl.org)

Пример команды:

```
openssl req -x509 -newkey rsa:2048 -nodes -keyout
certs/server.key -out certs/server.crt -days 365 -subj
"/CN=localhost"
```

Эта команда создаст:

- `certs/server.key` — приватный ключ;
- `certs/server.crt` — сертификат.

В локальной учебной среде браузер, скорее всего, будет предупреждать, что сертификат не доверенный. Это нормально для self-signed certificate.

Шаг 5. Конфигурация приложения

Создайте файл `internal/config/config.go`:

```
package config

type Config struct {
    Addr      string
    CertFile  string
    KeyFile   string
    DSN       string
}

func New() Config {
    return Config{
        Addr:      ":8443",
        CertFile:  "certs/server.crt",
        KeyFile:   "certs/server.key",
        DSN:
"postgres://postgres:postgres@localhost:5432/study_security
?sslmode=disable",
    }
}
```

В реальном проекте DSN, пути к сертификатам и адрес сервера лучше получать из переменных окружения, а не хардкодить.

Шаг 6. Модель студента

Создайте файл `internal/student/model.go`:

```
package student

type Student struct {
    ID          int64   `json:"id"`
    FullName    string  `json:"full_name"`
    StudyGroup  string  `json:"study_group"`
    Email       string  `json:"email"`
}
```

Шаг 7. Репозиторий для работы с БД

Создайте файл `internal/student/repo.go`:

```
package student

import (
    "database/sql"
    "errors"
)

var ErrStudentNotFound = errors.New("student not found")

type Repo struct {
    db *sql.DB
}

func NewRepo(db *sql.DB) *Repo {
    return &Repo{db: db}
```

```

    }

    // Опасный пример — так делать нельзя.
    func (r *Repo) UnsafeGetByID(rawID string) (*Student,
error) {
        query := "SELECT id, full_name, study_group, email
FROM students WHERE id = " + rawID

        row := r.db.QueryRow(query)

        var st Student
        err := row.Scan(&st.ID, &st.FullName, &st.StudyGroup,
&st.Email)
        if err != nil {
            if errors.Is(err, sql.ErrNoRows) {
                return nil, ErrStudentNotFound
            }
            return nil, err
        }

        return &st, nil
    }

    // Безопасный вариант через параметр.
    func (r *Repo) GetByID(id int64) (*Student, error) {
        row := r.db.QueryRow(
            "SELECT id, full_name, study_group, email FROM
students WHERE id = $1",
            id,
        )

        var st Student

```

```

    err := row.Scan(&st.ID, &st.FullName, &st.StudyGroup,
&st.Email)
    if err != nil {
        if errors.Is(err, sql.ErrNoRows) {
            return nil, ErrStudentNotFound
        }
        return nil, err
    }

    return &st, nil
}

func (r *Repo) PrepareGetByID() (*sql.Stmt, error) {
    return r.db.Prepare("SELECT id, full_name,
study_group, email FROM students WHERE id = $1")
}

```

Пояснение

Небезопасный пример показан только как демонстрация проблемы. Go-документация прямо предупреждает, что сбор SQL через форматирование строки небезопасен и создаёт SQL injection risk. Безопасный формат — передавать SQL и параметр отдельно, а placeholder syntax зависит от драйвера; для PostgreSQL это \$1. ([Go](#))

Шаг 8. HTTP-обработчики

Создайте файл `internal/httpapi/handler.go`:

```

package httpapi

import (
    "database/sql"
    "encoding/json"

```

```

    "net/http"
    "strconv"

    "example.com/pz5-security/internal/student"
)

type Handler struct {
    repo *student.Repo
    stmt *sql.Stmt
}

func NewHandler(repo *student.Repo, stmt *sql.Stmt)
*Handler {
    return &Handler{
        repo: repo,
        stmt: stmt,
    }
}

func (h *Handler) Health(w http.ResponseWriter, r
*http.Request) {
    if r.Method != http.MethodGet {
        http.Error(w, "method not allowed",
http.StatusMethodNotAllowed)
        return
    }

    w.Header().Set("Content-Type", "application/json;
charset=utf-8")
    _ = json.NewEncoder(w).Encode(map[string]string{
        "status": "ok",
        "scheme": "https",

```

```

    })
}

func (h *Handler) GetStudentByID(w http.ResponseWriter,
r *http.Request) {
    if r.Method != http.MethodGet {
        http.Error(w, "method not allowed",
http.StatusMethodNotAllowed)
        return
    }

    rawID := r.URL.Query().Get("id")
    if rawID == "" {
        http.Error(w, "id is required",
http.StatusBadRequest)
        return
    }

    id, err := strconv.ParseInt(rawID, 10, 64)
    if err != nil || id <= 0 {
        http.Error(w, "invalid id", http.StatusBadRequest)
        return
    }

    var st student.Student
    err = h.stmt.QueryRow(id).Scan(&st.ID, &st.FullName,
&st.StudyGroup, &st.Email)
    if err != nil {
        if err == sql.ErrNoRows {
            http.Error(w, "student not found",
http.StatusNotFound)
            return

```

```

    }
    http.Error(w, "internal server error",
http.StatusInternalServerError)
    return
}

w.Header().Set("Content-Type", "application/json;
charset=utf-8")
_ = json.NewEncoder(w).Encode(st)
}

```

Пояснение

Go рекомендует prepared statements для повторно используемых SQL-операций. `DB.Prepare` создаёт `sql.Stmt`, который затем можно вызывать через `Stmt.QueryRow`, `Stmt.Query` или `Stmt.Exec`, а после завершения работы statement нужно закрывать через `stmt.Close()`. (Go)

Шаг 9. Точка входа приложения

Создайте файл `cmd/server/main.go`:

```

package main

import (
    "crypto/tls"
    "database/sql"
    "log"
    "net/http"

    _ "github.com/lib/pq"

    "example.com/pz5-security/internal/config"

```

```
"example.com/pz5-security/internal/httpapi"
"example.com/pz5-security/internal/student"
)

func main() {
    cfg := config.New()

    db, err := sql.Open("postgres", cfg.DSN)
    if err != nil {
        log.Fatal(err)
    }
    defer db.Close()

    if err := db.Ping(); err != nil {
        log.Fatal(err)
    }

    repo := student.NewRepo(db)

    stmt, err := repo.PrepareGetByID()
    if err != nil {
        log.Fatal(err)
    }
    defer stmt.Close()

    handler := httpapi.NewHandler(repo, stmt)

    mux := http.NewServeMux()
    mux.HandleFunc("/health", handler.Health)
    mux.HandleFunc("/students", handler.GetStudentByID)
```



```

    cert, err := tls.LoadX509KeyPair(cfg.CertFile,
cfg.KeyFile)
    if err != nil {
        log.Fatal(err)
    }

    tlsConfig := &tls.Config{
        Certificates: []tls.Certificate{cert},
    }

    server := &http.Server{
        Addr:      cfg.Addr,
        Handler:   mux,
        TLSConfig: tlsConfig,
    }

    log.Println("HTTPS server started on
https://localhost:8443")

    err = server.ListenAndServeTLS("", "")
    if err != nil {
        log.Fatal(err)
    }
}

```

Пояснение

`tls.LoadX509KeyPair` загружает сертификат и ключ из PEM-файлов. После этого они помещаются в `tls.Config`. Документация `crypto/tls` прямо описывает `LoadX509KeyPair(certFile, keyFile)` как чтение и парсинг public/private key pair из файлов, содержащих PEM encoded data. ([pkg.go.dev](https://pkg.go.dev/crypto/tls))

В варианте выше используется `server.ListenAndServeTLS("", "")`, потому что сертификат уже передан через `TLSConfig`.

Шаг 10. Запуск приложения

В корне проекта выполните:

```
go run ./cmd/server
```

Ожидаемый результат:

```
HTTPS server started on https://localhost:8443
```

Шаг 11. Проверка HTTPS-маршрута

Проверьте в браузере:

```
https://localhost:8443/health
```

Ожидаемый ответ:

```
{"status":"ok","scheme":"https"}
```

Из-за self-signed certificate браузер может показать предупреждение безопасности. Для учебной локальной среды это ожидаемо.

Также можно проверить через curl:

```
curl -k https://localhost:8443/health
```

Флаг `-k` нужен, чтобы curl не отклонял самоподписанный сертификат.

Шаг 12. Проверка безопасного запроса к БД

Выполните:

```
curl -k "https://localhost:8443/students?id=1"
```

Ожидаемый ответ:

```
{
  "id": 1,
  "full_name": "Иванов Иван Иванович",
  "study_group": "ИБВО-01-25",
  "email": "ivanov@example.com"
}
```

Проверьте также:

```
curl -k "https://localhost:8443/students?id=999"
```

Ожидается 404 Not Found.

Шаг 13. Демонстрация риска SQL-инъекции

Теперь нужно разобрать, почему небезопасный вариант опасен.

Опасный подход:

```
query := "SELECT id, full_name, study_group, email FROM
students WHERE id = " + rawID
```

Если пользователь передаст не просто число, а фрагмент SQL, он может повлиять на структуру запроса. Go-документация показывает аналогичный небезопасный пример через `fmt.Sprintf`, после чего может получиться запрос вида `SELECT * FROM user WHERE id = 1 OR 1=1;`. OWASP отдельно указывает, что причиной SQL injection становятся *dynamic queries with string concatenation and user-supplied input*. ([Go](#))

Небезопасный пример

```
rows, err := db.Query(fmt.Sprintf("SELECT * FROM
students WHERE id = %s", rawID))
```

Так делать нельзя.

Безопасный пример

```
row := db.QueryRow(
```

```
"SELECT id, full_name, study_group, email FROM
students WHERE id = $1",
    id,
)
```

Именно такой подход рекомендует Go-документация: передавать SQL parameter values as function arguments, чтобы sql package превращал запрос в prepared statement и передавал параметр отдельно. (Go)

Шаг 14. Демонстрация prepared statement

В Go можно заранее подготовить SQL-запрос и затем многократно использовать его.

Пример:

```
stmt, err := db.Prepare("SELECT id, full_name,
study_group, email FROM students WHERE id = $1")
if err != nil {
    return err
}
defer stmt.Close()

row := stmt.QueryRow(1)
```

Go-документация объясняет, что prepared statement is SQL that is parsed and saved by the DBMS, а DB.Prepare возвращает sql.Stmt, который затем можно вызывать повторно. Также документация отдельно рекомендует закрывать statement через stmt.Close(). (Go)

Что получилось в результате

В ходе практической работы:

- подняли backend-сервер по HTTPS;
- подключили TLS-сертификат и приватный ключ;
- протестировали защищённый маршрут;
- разобрали природу SQL-инъекций;
- увидели небезопасный пример построения SQL;
- переписали запрос в безопасном виде через параметризацию;
- использовали prepared statement в Go.

Что важно понять по итогам работы

Безопасность backend-приложения начинается не с «больших» корпоративных решений, а с базовых инженерных практик.

Первая из них – не публиковать приложение в незашифрованном виде там, где нужна защита канала передачи данных. HTTPS и TLS решают эту задачу на транспортном уровне. Go предоставляет встроенные механизмы для запуска HTTPS-сервера и загрузки сертификатов из PEM-файлов. (pkg.go.dev)

Вторая – не допускать SQL-инъекций через динамическую сборку запросов. OWASP рекомендует prepared statements / parameterized queries как основную защиту, а Go-документация прямо предупреждает не собирать SQL через форматирование строк и отдельно указывает на безопасную передачу параметров в функции пакета `sql`. ([Cheat Sheet Series](#))

Дополнительные задания

Вариант 1. Добавить HTTPS-редирект

Реализуйте отдельный HTTP-сервер на порту 8080, который будет перенаправлять все запросы на `https://localhost:8443`.

Вариант 2. Вынести конфигурацию в переменные окружения

Перенесите Addr, DSN, CertFile, KeyFile в environment variables.

Вариант 3. Добавить метод поиска по email

Реализуйте маршрут:

```
GET /students/by-email?email=...
```

и выполните запрос только через параметризованный SQL.

Вариант 4. Добавить allow-list валидацию

Сделайте дополнительную проверку допустимого формата email или идентификатора до обращения к базе.

OWASP относит allow-list input validation к основным защитам наряду с prepared statements. ([Cheat Sheet Series](#))

Контрольные вопросы

1. Чем HTTP отличается от HTTPS?
2. Какую роль выполняет TLS в защищённом соединении?
3. Что такое TLS-сертификат?
4. Что делает `tls.LoadX509KeyPair`? ([pkg.go.dev](#))
5. Почему self-signed certificate подходит для локальной учебной среды, но не для публичного production-сервиса? [Вывод]
6. Что такое SQL-инъекция?
7. Почему конкатенация строки SQL с пользовательским вводом опасна? ([Cheat Sheet Series](#))
8. Что такое parameterized query?
9. Что такое prepared statement? ([Go](#))
10. Почему placeholder syntax может отличаться в разных СУБД и драйверах? ([Go](#))

Типичные ошибки

- Если сервер не запускается, сначала проверьте, существуют ли файлы `server.crt` и `server.key`, а также совпадают ли пути к ним.
- Если `tls.LoadX509KeyPair` возвращает ошибку, это обычно означает проблему с PEM-файлами сертификата или ключа. Документация указывает, что оба файла должны содержать PEM encoded data. ([pkg.go.dev](https://pkg.go.dev/crypto/tls))
- Если браузер показывает предупреждение безопасности, это нормально для самоподписанного сертификата в учебной локальной среде. [Вывод]
- Если SQL-запрос написан через конкатенацию строки, работа считается небезопасной даже тогда, когда «вроде бы передаётся только число». OWASP и Go-документация рассматривают сам принцип динамической сборки запроса из пользовательского ввода как источник SQL injection risk. ([Cheat Sheet Series](#))
- Если prepared statement создан, но не закрыт, это плохая практика. Go-документация рекомендует вызывать `stmt.Close()`, когда statement больше не нужен. ([Go](#))